

Name: Kabivalavan K

Register No: 727823tuit060

Class & Section: IV IT - E Course Name: Explainable AI Course Code: 231T010

GitHub Link: <https://github.com/Kabivalavan>

Explainable AI Assignment: Model Prediction and Explainability

1. Introduction

In modern machine learning, prediction accuracy alone is not sufficient. It is also important to understand why a machine learning model makes a particular prediction. Explainable Artificial Intelligence (XAI) provides techniques that make machine learning models more transparent and understandable.

For this assignment, neonatal child health risk prediction is considered as the application. The system uses clinical and neonatal characteristics such as gestational age, birth weight, Apgar scores, oxygen saturation, heart rate, respiratory rate, temperature, bilirubin, feeding difficulty, delivery mode and oxygen support to predict whether a neonatal profile is high risk.

A Random Forest Classifier is used to learn relationships between the neonatal characteristics and the high-risk target. SHAP and LIME are integrated to explain the prediction. The explanations show which features influence the model output for individual observations.

The dataset used in this demonstration is synthetic and is intended only for academic learning. It is not a clinical decision-support system.

2. Objective

1. To preprocess a neonatal child-health dataset containing clinical and demographic features.
2. To train a Random Forest Classifier for neonatal high-risk prediction.
3. To predict the risk class and probability for a neonatal profile.
4. To evaluate the performance of the trained model.
5. To integrate SHAP for global and local model explanation.
6. To integrate LIME-style local surrogate explanation for an individual prediction.
7. To visualize the feature contributions using SHAP and LIME plots.

3. XAI Explainability Tools Utilized

Tool Name: SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-agnostic Explanations). SHAP uses Shapley-value based feature attribution, while LIME builds a simple local surrogate model around one observation. Together they provide complementary explanations of the Random Forest prediction.

1. Positive SHAP contribution: moves the prediction toward the high-risk class.
2. Negative SHAP contribution: moves the prediction away from the high-risk class.
3. Large absolute SHAP value: stronger influence on the prediction.
4. LIME weight: indicates the local direction of influence in the surrogate model.

4. Dataset Description

The neonatal dataset contains synthetic records representing common neonatal health measurements and categorical descriptors. The `high_risk` column is the target variable. Numeric variables are standardized and categorical variables are converted to numerical representations using one-hot encoding.

1. `gestational_age_weeks` – gestational age in weeks
2. `birth_weight_kg` – birth weight in kilograms
3. `apgar_1min` and `apgar_5min` – Apgar scores
4. `heart_rate_bpm` – heart rate
5. `respiratory_rate_bpm` – respiratory rate
6. `spo2_percent` – oxygen saturation
7. `temperature_c` – body temperature
8. `bilirubin_mg_dl` – bilirubin level
9. `feeding_difficulty` – feeding difficulty indicator
10. `sex` – neonatal sex
11. `delivery_mode` – normal or cesarean
12. `oxygen_support` – level of respiratory support
13. `high_risk` – target class (0 = lower risk, 1 = high risk)

5. Python Program – Installation

```
!pip install pandas numpy matplotlib seaborn scikit-learn shap lime joblib streamlit
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
```

5. Python Program – Load Dataset

```
df = pd.read_csv("data/neonatal_health_synthetic.csv")  
print(df.head())  
print(df.shape)  
print(df.isnull().sum())
```

5. Python Program – Feature Selection

```
X = df.drop(columns=["high_risk"])
y = df["high_risk"]

numeric_features = X.select_dtypes(include=["number"]).columns.tolist()
categorical_features = X.select_dtypes(exclude=["number"]).columns.tolist()

print(numeric_features)
print(categorical_features)
```

5. Python Program – Preprocessing

```
preprocessor = ColumnTransformer([
    ("num", StandardScaler(), numeric_features),
    ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False),
     categorical_features)
])
```


5. Python Program – Random Forest Model

```
model = RandomForestClassifier(  
    n_estimators=300,  
    max_depth=10,  
    min_samples_leaf=3,  
    class_weight="balanced",  
    random_state=42  
)  
  
pipeline = Pipeline([  
    ("preprocessor", preprocessor),  
    ("model", model)  
])
```

5. Python Program – Training

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42
)
pipeline.fit(X_train, y_train)

y_pred = pipeline.predict(X_test)
y_prob = pipeline.predict_proba(X_test)[:, 1]
```

5. Python Program – Prediction

```
print("Predicted classes:", y_pred[:10])  
print("High-risk probabilities:", y_prob[:10])
```

```
results = X_test.copy()  
results["actual"] = y_test.values  
results["predicted"] = y_pred  
results["risk_probability"] = y_prob
```

5. Python Program – Save Model

```
import joblib
joblib.dump(pipeline, "src/neonatal_risk_model.joblib")
print("Model saved successfully.")
```

6. Explainability Program – SHAP Import

```
import shap

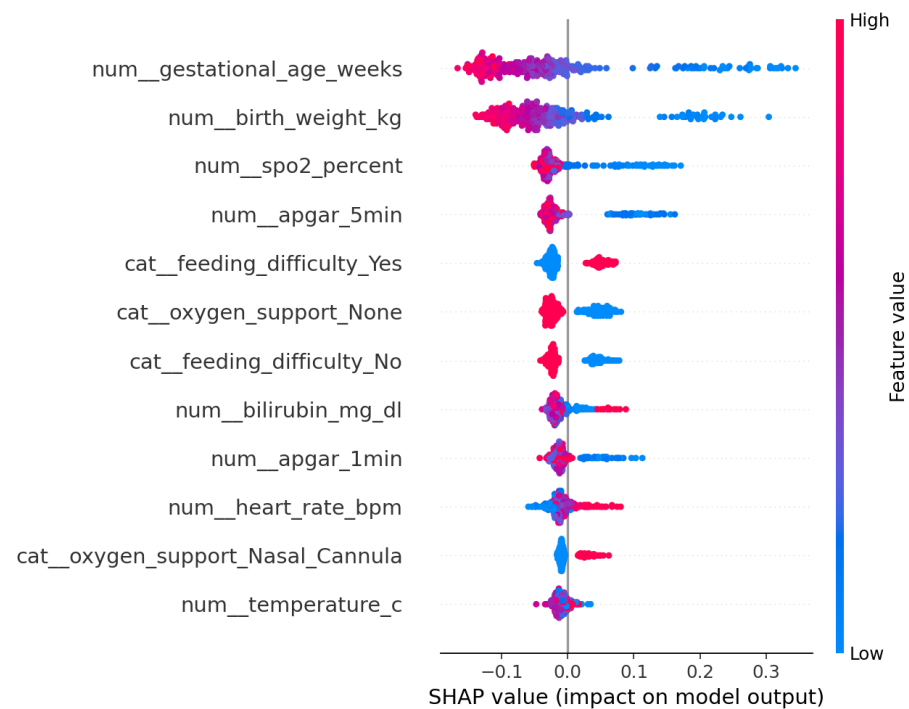
explainer = shap.TreeExplainer(
    pipeline.named_steps["model"]
)
X_test_transformed = pipeline.named_steps["preprocessor"].transform(X_test)
shap_values = explainer.shap_values(X_test_transformed)
```

6. Explainability Program – Feature Names

```
feature_names = pipeline.named_steps["preprocessor"].get_feature_names_out()
feature_names = [n.replace("num__", "").replace("cat__", "") for n in feature_names]
print(feature_names)
```

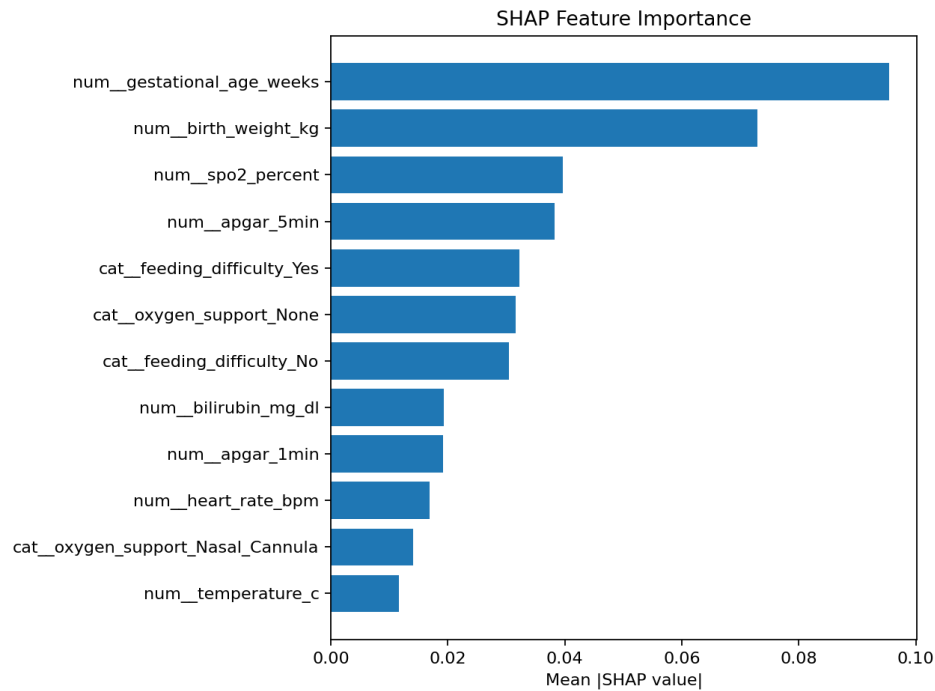
6. Explainability Program – SHAP Summary

The SHAP summary plot ranks the transformed features according to their contribution across the test observations.



6. Explainability Program – SHAP Importance

The SHAP bar chart displays mean absolute SHAP values. Higher values indicate stronger average influence on model predictions.



6. Explainability Program – Local SHAP

```
instance = X_test.iloc[[0]]
instance_t = pipeline.named_steps["preprocessor"].transform(instance)
instance_values = explainer.shap_values(instance_t)
print("Prediction:", pipeline.predict(instance)[0])
print("Probability:", pipeline.predict_proba(instance)[0, 1])
```

6. Explainability Program – LIME Import

```
from lime.lime_tabular import LimeTabularExplainer

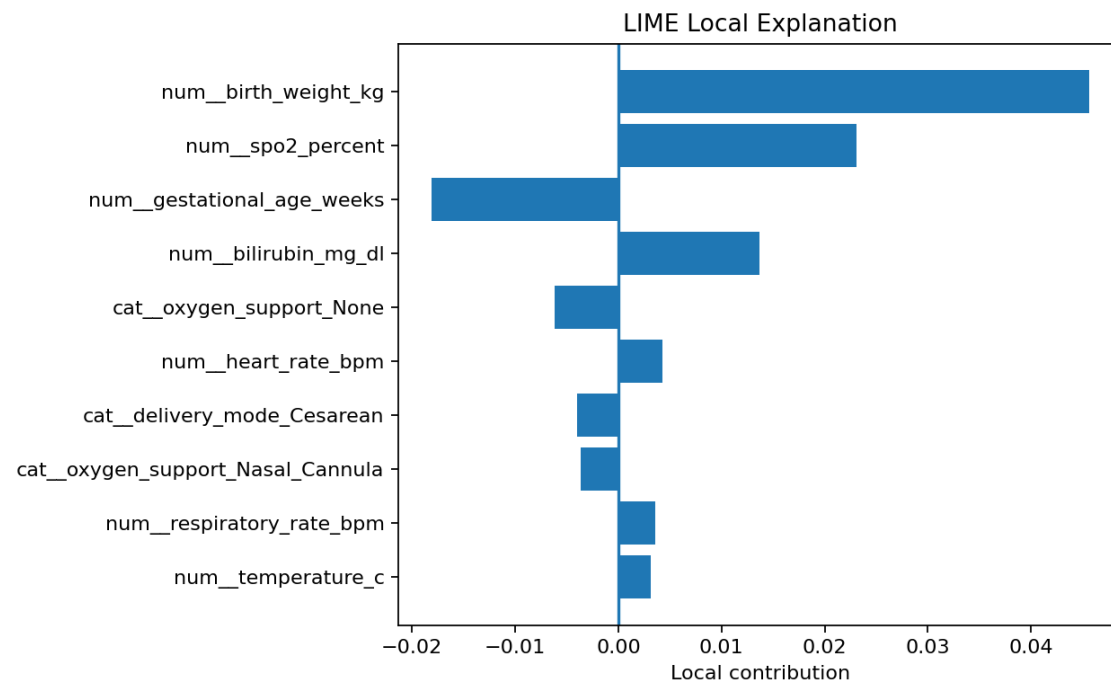
lime_explainer = LimeTabularExplainer(
    training_data=X_train_transformed,
    mode="classification",
    feature_names=feature_names,
    class_names=["Lower risk", "High risk"],
    random_state=42
)
```

6. Explainability Program – LIME Explanation

```
lime_exp = lime_explainer.explain_instance(  
    X_test_transformed[0],  
    pipeline.named_steps["model"].predict_proba,  
    num_features=10  
)  
print(lime_exp.as_list())
```

6. Explainability Program – LIME Local Plot

The LIME local explanation uses a simple surrogate approximation around one neonatal observation. Positive contributions support the selected class and negative contributions oppose it.



7. Model Used

Random Forest Classifier A Random Forest Classifier is used for neonatal high-risk prediction. Random Forest combines multiple decision trees and aggregates their outputs to obtain a robust classification. Since the target variable is categorical, classification is appropriate.

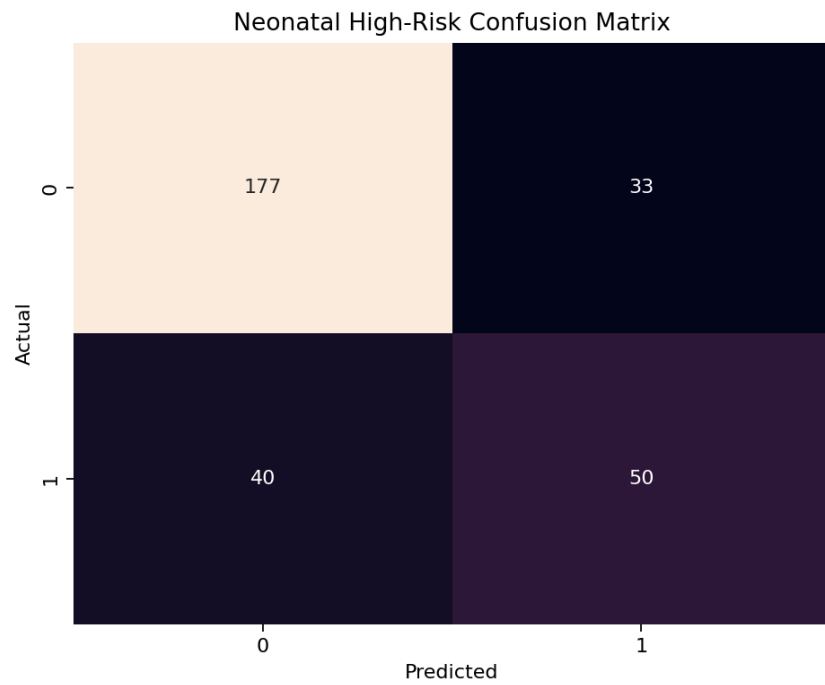
1. Gestational age
2. Birth weight
3. Apgar scores
4. Oxygen saturation
5. Heart rate
6. Respiratory rate
7. Temperature
8. Bilirubin
9. Feeding difficulty
10. Delivery mode
11. Oxygen support

8. Model Evaluation

The model is evaluated using Accuracy, Precision, Recall, F1-score and ROC-AUC. These measures provide a more complete view than accuracy alone, especially when the high-risk class is less frequent.

1. Accuracy = 0.7567
2. Precision = 0.6024
3. Recall = 0.5556
4. F1-score = 0.5780
5. ROC-AUC = 0.7948

8. Model Evaluation – Confusion Matrix



8. Model Evaluation – Interpretation

Accuracy measures the proportion of correct predictions. Precision measures how many predicted high-risk profiles were actually high risk. Recall measures how many actual high-risk profiles were identified. F1-score balances precision and recall. ROC-AUC summarizes ranking performance across probability thresholds.

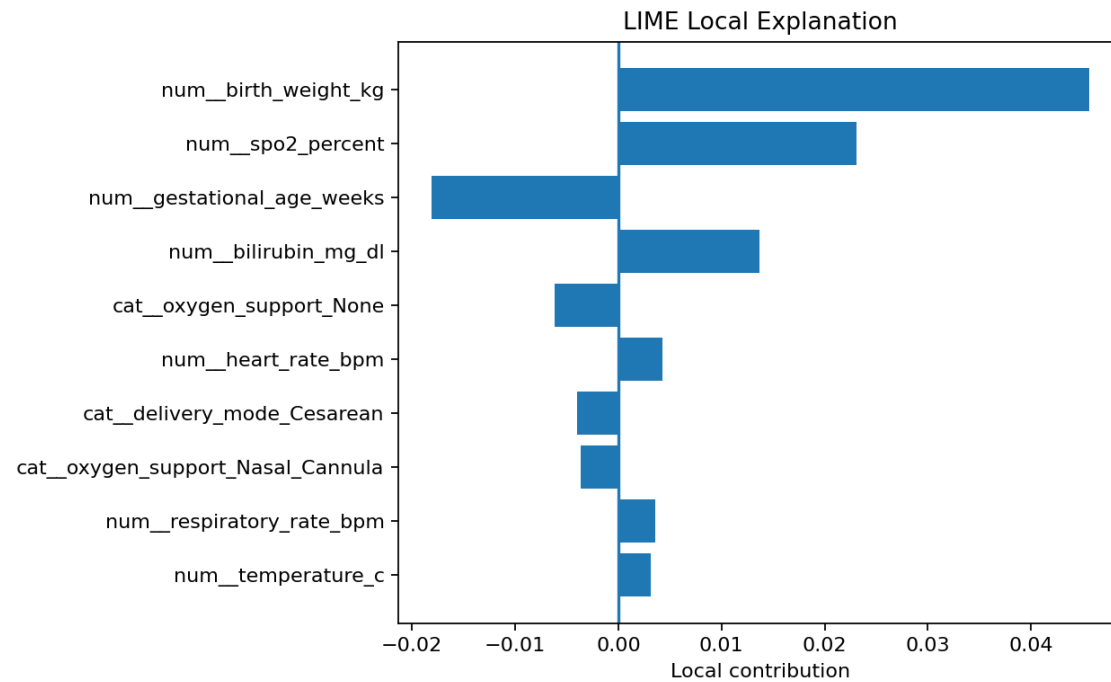
9. Individual Neonatal Risk Prediction

After training the model, one neonatal profile from the test dataset is selected. The model predicts both a class and a high-risk probability.

```
example = X_test.iloc[[0]]
print(example)
prediction = pipeline.predict(example)[0]
probability = pipeline.predict_proba(example)[0, 1]
print("Predicted class:", prediction)
print("High-risk probability:", probability)
```

9. Individual Neonatal Risk Prediction – Output

Example predicted high-risk probability: 0.1012 Example predicted class: Lower risk



9. Individual Neonatal Risk Prediction – SHAP Interpretation

SHAP explains the individual prediction by showing which transformed features push the Random Forest output toward or away from the high-risk class. The magnitude of the contribution indicates the strength of the effect for that observation.

9. Individual Neonatal Risk Prediction – LIME Interpretation

LIME approximates the local behavior of the classifier with a simpler surrogate model. It is useful for understanding the most influential local conditions around a single neonatal observation.

9. Individual Neonatal Risk Prediction – Example Record

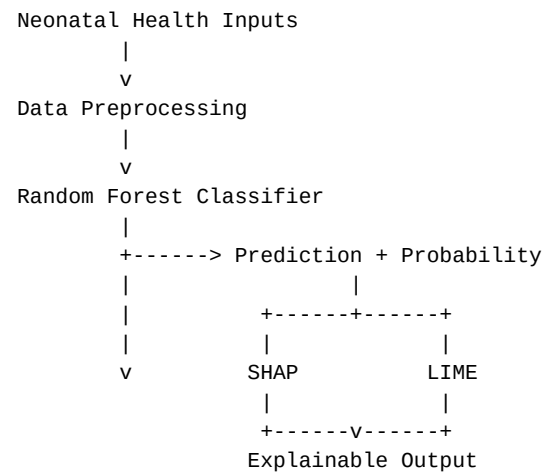
gestational_age_weeks	birth_weight_kg	apgar_1min	apgar_5min	heart_rate_bpm	respiratory_rate_bpm	spo2_percent	t..
37.7	3.67	7	8	155	50	99	..

10. SHAP vs LIME

Both methods aim to make model predictions easier to interpret, but they use different principles.

1. SHAP uses Shapley-value feature attribution and can provide global and local explanations.
2. LIME creates a local surrogate model around a selected observation.
3. SHAP is particularly efficient here because the prediction model is tree-based.
4. LIME is model-agnostic and focuses on local interpretability.
5. Using both methods provides a complementary view of the same prediction.

11. Project Architecture



12. Advantages of Explainability

1. Improves transparency of machine-learning predictions.
2. Helps identify influential neonatal features.
3. Supports model debugging and error analysis.
4. Makes model behavior easier to communicate and audit.
5. Provides local explanations for individual predictions.

13. Limitations

1. The dataset is synthetic and does not represent real patients.
2. The target is generated for educational demonstration.
3. The model has not been clinically validated.
4. SHAP and LIME explain model behavior; they do not prove medical causation.
5. Clinical deployment would require approved datasets, validation, privacy controls and clinical oversight.

14. Future Enhancements

1. Use an approved de-identified neonatal dataset.
2. Compare Random Forest with calibrated and gradient-boosting models.
3. Add cross-validation and hyperparameter optimization.
4. Add ROC and precision-recall curves.
5. Perform subgroup and fairness analysis.
6. Integrate an official LimeTabularExplainer in the application.
7. Add model monitoring and versioning.

15. Streamlit Application

```
streamlit run app.py
```

```
# app.py
```

```
st.title("Neonatal Health Risk Prediction using XAI")
```

```
if st.button("Predict and Explain"):
```

```
    probability = model.predict_proba(instance)[0, 1]
```

```
    st.metric("High-risk probability", f"{probability:.1%}")
```

```
    # Display SHAP and LIME explanations
```

16. Model Saving and Reproducibility

```
joblib.dump(  
    pipeline,  
    "src/neonatal_risk_model.joblib"  
)
```

```
# Results are saved as CSV and PNG files  
# Random state = 42 for reproducibility
```

17. GitHub Project Structure

```
neonatal_health_shap_lime_project/  
|-- data/neonatal_health_synthetic.csv  
|-- src/train_model.py  
|-- src/explain_model.py  
|-- src/lime_local.py  
|-- src/neonatal_risk_model.joblib  
|-- results/shap_summary.png  
|-- results/shap_feature_importance.png  
|-- results/lime_local_explanation.png  
|-- results/confusion_matrix.png  
|-- results/test_predictions.csv  
|-- app.py  
|-- requirements.txt  
|-- README.md
```

18. GitHub Submission

GitHub Profile: <https://github.com/Kabivalavan> Recommended project repository: <https://github.com/Kabivalavan/neonatal-health-shap-lime>

```
git init
git add .
git commit -m "Neonatal SHAP LIME XAI project"
git branch -M main
git remote add origin https://github.com/Kabivalavan/neonatal-health-shap-lime.git
git push -u origin main
```

19. Conclusion

This assignment implements a complete explainable machine-learning pipeline for neonatal child-health risk prediction. A Random Forest Classifier is trained on synthetic neonatal features, evaluated using standard classification metrics, and explained using SHAP and LIME. The final system demonstrates how a prediction can be accompanied by feature-level explanations rather than presenting only a class label.

20. References

1. Lundberg, S. M. & Lee, S.-I. A Unified Approach to Interpreting Model Predictions. NeurIPS, 2017.
2. Ribeiro, M. T., Singh, S. & Guestrin, C. Why Should I Trust You? Explaining the Predictions of Any Classifier. KDD, 2016.
3. scikit-learn documentation – Random Forest and model evaluation.
4. SHAP documentation – TreeExplainer and model explanations.
5. LIME documentation – LimeTabularExplainer.
6. World Health Organization – neonatal health and mortality indicators.

21. Submission Information

Name: Kabivalavan K Register No: 727823tuit060 Class & Section: IV IT - E Course Name: Explainable AI Course Code: 231T010 GitHub Link: <https://github.com/Kabivalavan>

1. Project Title: Implement SHAP and LIME using the ML model – Child Health in Neonatal
2. Model: Random Forest Classifier
3. Explainability: SHAP + LIME
4. Dataset: Synthetic neonatal child-health dataset

Appendix – Project Code and Results (20)

This appendix provides reproducibility details, code fragments, generated outputs and implementation notes for the neonatal SHAP and LIME project.

```
python src/train_model.py  
python src/explain_model.py  
streamlit run app.py
```

See results/ for generated plots and prediction tables.

Appendix – Project Code and Results (21)

This appendix provides reproducibility details, code fragments, generated outputs and implementation notes for the neonatal SHAP and LIME project.

```
python src/train_model.py  
python src/explain_model.py  
streamlit run app.py
```

```
# See results/ for generated plots and prediction tables.
```

Appendix – Project Code and Results (22)

This appendix provides reproducibility details, code fragments, generated outputs and implementation notes for the neonatal SHAP and LIME project.

```
python src/train_model.py  
python src/explain_model.py  
streamlit run app.py
```

```
# See results/ for generated plots and prediction tables.
```

Appendix – Project Code and Results (23)

This appendix provides reproducibility details, code fragments, generated outputs and implementation notes for the neonatal SHAP and LIME project.

```
python src/train_model.py  
python src/explain_model.py  
streamlit run app.py
```

See results/ for generated plots and prediction tables.

Appendix – Project Code and Results (24)

This appendix provides reproducibility details, code fragments, generated outputs and implementation notes for the neonatal SHAP and LIME project.

```
python src/train_model.py  
python src/explain_model.py  
streamlit run app.py
```

```
# See results/ for generated plots and prediction tables.
```

Appendix – Project Code and Results (25)

This appendix provides reproducibility details, code fragments, generated outputs and implementation notes for the neonatal SHAP and LIME project.

```
python src/train_model.py  
python src/explain_model.py  
streamlit run app.py
```

```
# See results/ for generated plots and prediction tables.
```

Appendix – Project Code and Results (26)

This appendix provides reproducibility details, code fragments, generated outputs and implementation notes for the neonatal SHAP and LIME project.

```
python src/train_model.py  
python src/explain_model.py  
streamlit run app.py
```

```
# See results/ for generated plots and prediction tables.
```